11d. Thread-Safe Operations

Martin Alfaro
PhD in Economics

INTRODUCTION

Multithreading allows multiple threads to run simultaneously within a single process, enabling parallel execution of operations on the same machine. Unlike other forms of parallelization such as multiprocessing, where each process has its own memory space, **threads in multithreading share a common memory space**.

This shared-memory environment makes parallelization easy, but it also introduces some complexity while writing code: since all threads can access and modify the same data simultaneously, **parallel execution can lead to unintended side effects if not managed carefully**. In particular, the interaction between threads may cause corrupted results or even program crashes.

To reason about these issues, it's useful to distinguish between thread-safe and thread-unsafe operations. An operation is considered **thread-safe** if it can be executed in parallel without causing inconsistencies, crashes, or corrupted results. Conversely, **thread-unsafe** operations require explicit synchronization or algorithmic restructuring to prevent corruption.

The current section will identify key features that render operations unsafe under multithreading. In particular, we'll see that common operations like reductions aren't thread-safe, leading to incorrect results if multithreading is applied naively. We'll also explore the concept of embarrassingly parallel programs, which are a prime example of thread-safe operations. These programs can be divided into independent units of work, requiring no communication or synchronization. These types of programs can be parallelized directly, without the need to restructure their computation.

> ⚠️ **Warning!** - Loaded Package
>
> All the scripts below assume you've executed `using Base.Threads`. This allows direct access to macros like `@threads` and `@spawn`.
>
> Furthermore, we assume your Julia session is running with more than one thread available. Refer to these instructions to enable multithreading.

THREAD-UNSAFE OPERATIONS

In a multithreaded environment, an operation is deemed unsafe if executing it concurrently can lead to incorrect behavior, data corruption, or even program crashes. These issues typically arise when tasks exhibit some degree of dependency, either in terms of operations or shared resources. The examples below illustrate how these dependencies can introduce serious issues when multithreading is applied without proper safeguards.

WRITING ON A SHARED VARIABLE

One of the simplest examples of a thread-unsafe operation is writing to a shared variable. To illustrate, consider a scalar variable `output` initialized to zero. This is then updated in a for-loop with two iterations, where the *i*-th iteration sets `output` to the value `i`.

To starkly show the challenges of concurrent execution, we'll deliberately introduce a decreasing delay before each update of `output`. This is implemented with `sleep(1/i)`, causing the first iteration to pause for 1 second and the second iteration for half a second. Although this delay is artificially induced via `sleep`, it mimics the kind of timing gaps that real computations can introduce. Such gaps can cause `output` to be updated in an unexpected order.

SEQUENTIAL

```julia
function foo()
    output = 0

    for i in 1:2
        sleep(1/i)
        output = i
    end

    return output
end
```

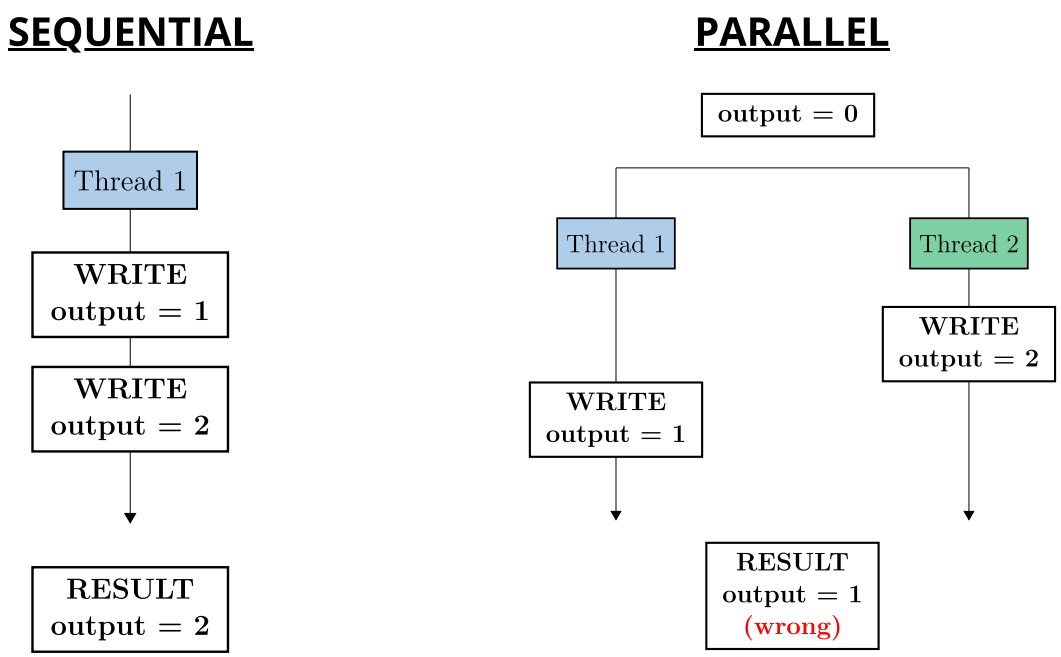
```
julia> foo()
2
```

PARALLEL

```julia
function foo()
    output = 0

    @threads for i in 1:2
        sleep(1/i)
        output = i
    end

    return output
end
```

```
julia> foo()
1
```

The delay is inconsequential for a sequential procedure, where `output` takes on the values 0, 1, and 2 as the program progresses. However, when executed concurrently, the first iteration completes after the second iteration has finished. As a result, the sequence of values for `output` is 0, 2, and 1.



While the problem may seem apparent in this simple example, it can manifest in more complex and subtle ways in real-world applications. The core issue is that the order of execution isn't guaranteed in a multithreaded environment. Thus, when multiple threads modify the same shared variable, the final value depends on which thread executes last.

In fact, the issue can be exacerbated when each iteration additionally involves reading a shared variable. Next, we consider such a scenario.

READING AND WRITING A SHARED VARIABLE

Reading and writing shared data doesn't necessarily yield incorrect results. For instance, a parallel for-loop could safely mutate a vector: even if multiple threads are simultaneously modifying the same shared object, each thread would be operating on distinct elements of the vector. Since no two threads would interfere with one another, the updates would remain independent.

Problems arise, however, **when the correctness of reading and writing shared data depends on the order in which threads execute**. This situation is known as a **race condition**. The term reflects that the final output may change from one run to the next, depending on which thread finishes and updates the data last.

To demonstrate the issue, let's modify our previous example by introducing a variable `temp`, whose value is updated in each iteration. This variable will be shared across threads and used to mutate the *i*-th entry of the vector `output`. By introducing a delay before writing each entry of `output`, a race condition is introduced, where all threads end up using the last value of `temp` (in this case, 2).

SEQUENTIAL

```julia
function foo()
    output = Vector{Int}(undef,2)
    temp   = 0

    for i in 1:2
        temp      = i; sleep(i)
        output[i] = temp
    end

    return output
end
```

```
julia> foo()
2-element Vector{Int64}:
 1
 2
```

PARALLEL (SHARED VARIABLE)

```julia
function foo()
    output = Vector{Int}(undef,2)
    temp   = 0

    @threads for i in 1:2
        temp      = i; sleep(i)
        output[i] = temp
    end

    return output
end
```

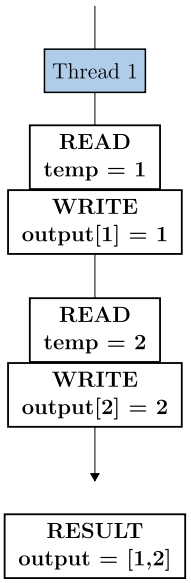
```
julia> foo()
2-element Vector{Int64}:
 2
 2
```

PARALLEL (NO SHARED VARIABLE)

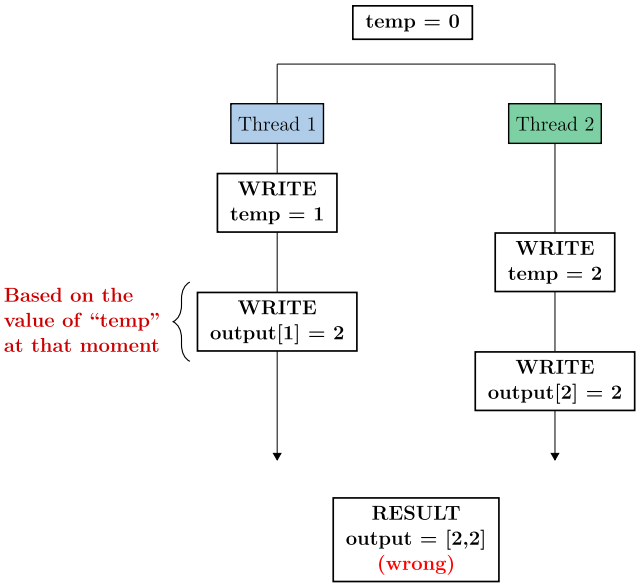
```julia
function foo()
    output = Vector{Int}(undef,2)


    @threads for i in 1:2
        temp      = i; sleep(i)
        output[i] = temp
    end

    return output
end
```

```julia
julia> foo()
2-element Vector{Int64}:
 1
 2
```

SEQUENTIAL



PARALLEL



In this specific scenario, the issue can be easily circumvented: instead of defining `temp` outside the for-loop, we simply define it as variable local to the for-loop. In this way, each thread would work with its own private copy of `temp`, thereby eliminating the data race entirely.

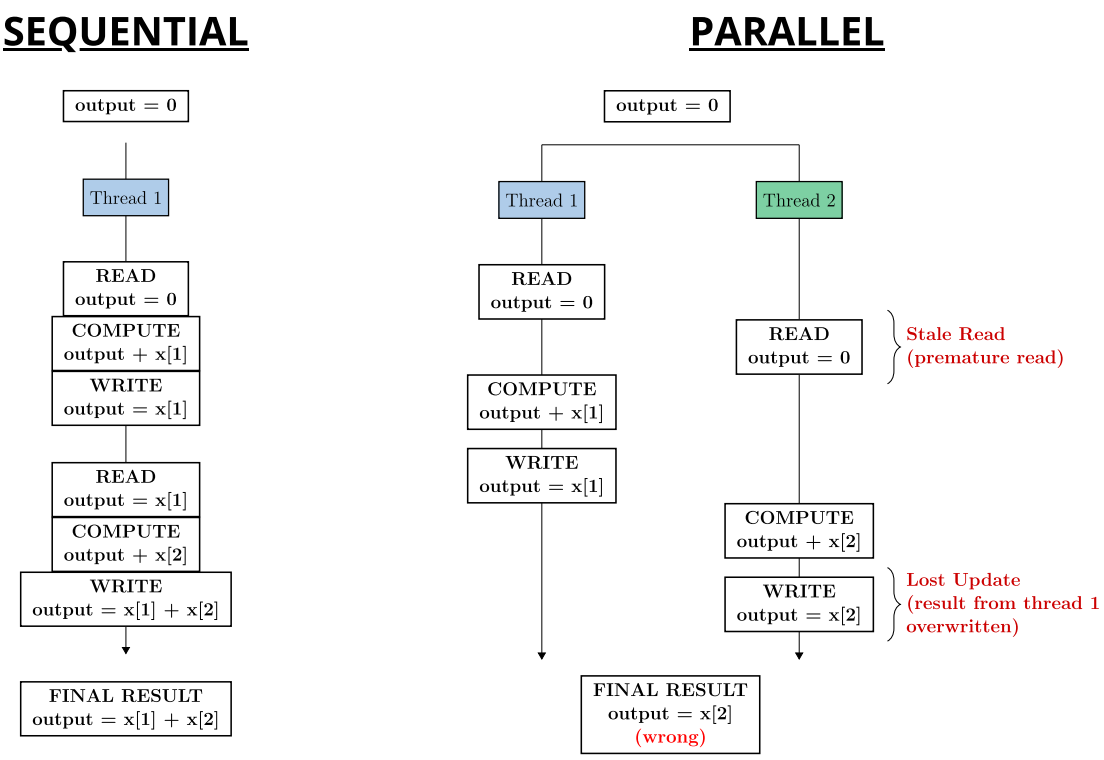
Beyond any solution proposed, the example highlights the subtleties of parallelizing operations. Even simple patterns can conceal dependencies that lead to unsafe behavior when executed concurrently. To illustrate this further, we now turn to a more common scenario where data races occur: reductions.

RACE CONDITIONS WITH REDUCTIONS

Reductions are a prime example of thread-unsafe operations. To illustrate why this is so, consider summing a collection in parallel. At first glance, this seems straightforward: each thread computes a partial contribution and adds it to a shared accumulator. However, the variable holding each partial sum is shared across all threads: during each iteration, every thread attempts to read its current value, add its contribution, and write the result back.

When multiple threads perform these steps concurrently, their actions can overlap in unpredictable ways. One thread's update may overwrite another's, meaning that some contributions are lost rather than combined. As a result, the final sum is nondeterministic and often incorrect, varying from run to run.

Below, we illustrate this issue when we sum the first two elements of a vector `x`.



When performing reductions in parallel without addressing the underlying race condition, the outcome becomes unpredictable. More specifically, the final sum tends to be lower than the correct value, because some updates are lost when threads overwrite one another's contributions.

SEQUENTIAL

```julia
x = rand(1_000_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output += x[i]
    end

    return output
end
```

```
julia> foo(x)
500658.01158503356
```

MULTITHREADED 1

```julia
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @threads for i in eachindex(x)
        output += x[i]
    end

    return output
end
```

```
julia> foo(x)
21436.48668413443
```

MULTITHREADED 2

```julia
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @threads for i in eachindex(x)
        output += x[i]
    end

    return output
end
```

```
julia> foo(x)
21590.997961948713
```

MULTITHREADED 3

```julia
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @threads for i in eachindex(x)
        output += x[i]
    end

    return output
end
```

```
julia> foo(x)
21461.273851717895
```

The key insight from this example isn't that reductions are incompatible with multithreading. Rather, it's that the strategy to apply multithreading needs to be adapted accordingly. The upcoming sections will present these strategies. Before that, we conclude this section by looking at the opposite end of the spectrum: problems that naturally lend themselves to parallel execution.

EMBARRASSINGLY PARALLEL PROGRAMS

In the context of thread-safe programming, the simplest case occurs when tasks are completely independent of one another. These cases are known as *embarrassingly parallel problems*. **embarrassingly parallel problems**. In these problems, the computation can be decomposed into many independent tasks, each of which can be executed in parallel without communication, synchronization, or shared state. Because no task influences any other, the program has full freedom to schedule and execute them in any order, making parallelization highly efficient.

In the context of for-loops, a simple way to parallelize embarrassingly parallel problems is through the macro `@threads` from the package `Threads`. This is a form of thread-based parallelism, where the distribution of work is based on the number of threads available. In particular, `@threads` attempts to balance the workload by dividing the iterations as evenly as possible. Unlike `@spawn`, `@threads` automatically schedules the tasks and waits for their completion before execution proceeds. In the next section, we'll present a detailed comparison between `@threads` and `@spawn`. For now, we simply demonstrate the simplicity of `@threads` through the following example.

NOT THREADED

```julia
x_small  = rand(    1_000)
x_medium = rand(  100_000)
x_big    = rand(1_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x_small)
  3.319 μs (3 allocations: 7.883 KiB)
julia> @btime foo($x_medium)
  332.609 μs (3 allocations: 781.320 KiB)
julia> @btime foo($x_big)
  3.396 ms (3 allocations: 7.629 MiB)
```

MULTITHREADED

```julia
x_small  = rand(    1_000)
x_medium = rand(  100_000)
x_big    = rand(1_000_000)

function foo(x)
    output = similar(x)

    @threads for i in eachindex(x)
        output[i] = log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x_small)
  9.092 μs (125 allocations: 20.508 KiB)
julia> @btime foo($x_medium)
  42.710 μs (125 allocations: 793.945 KiB)
julia> @btime foo($x_big)
  336.284 μs (125 allocations: 7.642 MiB)
```